Mode Opératoire : Rendre une Blazor Web App (.NET 8) disponible Hors-Ligne (PWA)

Ce guide détaille les étapes exactes pour ajouter des fonctionnalités hors-ligne et PWA (Progressive Web App) à un projet **Blazor Web App** fraîchement créé en **.NET 8** avec interactivité `Auto`, `WebAssembly` ou `Server`.

Prérequis

- Un projet Blazor Web App composé de deux parties : le projet Serveur (par exemple `MonApp`) et le projet Client (`MonApp.Client`).
- Le mode d'interactivité **WebAssembly** doit être activé pour les pages hors-ligne.

Étape 1 : Fichiers statiques et Manifeste PWA

Dans le **projet Client** (`MonApp.Client/wwwroot`), vous devez d'abord mettre en place les fichiers de base de la PWA.

1. **Ajouter des icônes** Ajoutez une favicon et des icônes adaptées à la PWA dans `wwwroot` (ex: `icon-192.png`, `icon-512.png`).

2. **Créer `manifest.json`** dans `wwwroot` :

```json
{
  "name": "Mon App Hors-Ligne",
  "short_name": "MonApp",
  "start_url": "./",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#03173d",
  "icons": [
    {
      "src": "icon-192.png",
      "type": "image/png",
      "sizes": "192x192"
    },
    {
      "src": "icon-512.png",
      "type": "image/png",
      "sizes": "512x512"
    }
  ]
}
```

3. **Lier le Manifeste dans l'interface globale** Dans le projet Serveur, ouvrez `Components/App.razor` (ou `_Host.cshtml`) et ajoutez ces balises dans le `<head>` :

```
<link href="manifest.json" rel="manifest" />
<link rel="apple-touch-icon" sizes="512x512" href="icon-512.png" />
<link rel="apple-touch-icon" sizes="192x192" href="icon-192.png" />
```

Étape 2 : Configuration du `.csproj` (Client)

Le manifeste des ressources WebAssembly (`service-worker-assets.js`) doit être généré à la compilation pour lister tous les `.dll` et fichiers statiques à mettre en cache.

1. Ouvrez `MonApp.Client.csproj` et ajoutez ceci dans un `<PropertyGroup>` :

```
<PropertyGroup>
  <ServiceWorkerAssetsManifest>service-worker-
assets.js</ServiceWorkerAssetsManifest>
</PropertyGroup>
```

2. Ajoutez la référence au Service Worker :

```
<ItemGroup>
  <ServiceWorker Include="wwwroot\service-worker.js"
PublishedContent="wwwroot\service-worker.published.js" />
</ItemGroup>
```

Étape 3 : Création des Services Workers

Créez ces deux fichiers dans le dossier `wwwroot` du projet **Client**.

1. `service-worker.js` (Pour le développement local)

En développement, on recharge à chaud, il vaut mieux importer le worker publié ou l'ignorer pour éviter de piéger votre navigateur sur une vieille version de code :

```
// On importe le script complet pour que le hors-ligne marche en dev,
// sinon laissez ce fichier vide pour éviter le cache intempestif !
self.importScripts('./service-worker.published.js');
```

2. `service-worker.published.js` (Stratégie de Cache Résiliente)

Ce fichier est le cœur du mode hors-ligne. Il télécharge tout. **Attention** : `cache.addAll()` échoue complètement si un seul fichier répond 404. Il faut donc une version **tolérante aux erreurs** pour la Blazor Web App !

```
self.importScripts('./service-worker-assets.js');
self.addEventListener('install', event =>
event.waitUntil(onInstall(event)));
self.addEventListener('activate', event =>
event.waitUntil(onActivate(event)));
self.addEventListener('fetch', event =>
event.respondWith(onFetch(event)));

const cacheNamePrefix = 'offline-cache-';
const cacheName = `${cacheNamePrefix}${self.assetsManifest.version}`;
const offlineAssetsInclude = [/\.dll$/, /\.pdb$/, /\.wasm/, /\.html/,
/\.js$/, /\.json$/, /\.css$/, /\.woff$/, /\.png$/, /\.jpe?g$/, /\.gif$/,
/\.ico$/, /\.blat$/, /\.dat$/];
const offlineAssetsExclude = [/^service-worker\.js$/];

async function onInstall(event) {
    console.info('Service worker: Install');

    // Mettre en cache les fichiers système (framework, dlls...)
    const assetsRequests = self.assetsManifest.assets
        .filter(asset => offlineAssetsInclude.some(pattern =>
pattern.test(asset.url)))
        .filter(asset => !offlineAssetsExclude.some(pattern =>
pattern.test(asset.url)))
        .map(asset => new Request(asset.url, { cache: 'no-cache' })); //
Retirer l'attribut 'integrity: asset.hash' pour éviter les erreurs SRI en
dev (incompatibilité dotnet watch)

    // Cacher explicitement les routes des pages hors-ligne (Important
pour la structure WebApp)
    assetsRequests.push(new Request('/', { cache: 'no-cache' }));
    assetsRequests.push(new Request('offline', { cache: 'no-cache' })); //
Remplacer par vos URLs

    await caches.open(cacheName).then(async cache => {
        // Enregistrement individuel manuel pour ne pas échouer au moindre
fichier "404" (résilience)
        for (const req of assetsRequests) {
            try {
                const response = await fetch(req);
                if (response.ok) {
                    await cache.put(req, response);
                } else {
                    console.warn(`SW: Échec de cache ${req.url} (Statut:
${response.status})`);
                }
            } catch (error) {
                console.warn(`SW: Erreur réseau au caching de ${req.url}`,
error);
            }
        }
    });
}
```

```
async function onActivate(event) {
    console.info('Service worker: Activate');
    const cacheKeys = await caches.keys();
    await Promise.all(cacheKeys
        .filter(key => key.startsWith(cacheNamePrefix) && key !==
cacheName)
        .map(key => caches.delete(key)));

    // Prendre le contrôle immédiat des clients (navigateurs) sans
nécessiter de rechargement page
    await self.clients.claim();
}

async function onFetch(event) {
    let cachedResponse = null;

    // Stratégie pour les navigations web intégrales (changement de pages)
    if (event.request.mode === 'navigate') {
        event.respondWith(
            (async () => {
                try {
                    // Essayer toujours d'avoir le réseau en premier pour
les pages HTML
                    const networkResponse = await fetch(event.request);
                    return networkResponse;
                } catch (error) {
                    // Si réseau en rade => rechercher la route exacte
dans le cache
                    const cache = await caches.open(cacheName);
                    cachedResponse = await cache.match(event.request);
                    if (cachedResponse) return cachedResponse;

                    // Fallback générique : On rend la racine de
l'application
                    return await cache.match('/');
                }
            })()
        );
        return;
    }

    // Stratégie pour les autres ressources (GET webassembly, JS, CSS)
    if (event.request.method === 'GET') {
        const cache = await caches.open(cacheName);
        cachedResponse = await cache.match(event.request);
    }

    // On retourne le cache ou bien directement le nouveau fichier du
réseau
    return cachedResponse || fetch(event.request);
}
```

Étape 4 : Enregistrement du Service Worker

Dans `Components/App.razor` (côté Serveur), insérez ce script juste avant la fermeture de la balise `</body>`. Il s'assure que le navigateur détecte et installe le `service-worker.js`.

```
<script>
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/service-worker.js')
            .then(() => console.log('Service Worker enregistré.'))
            .catch(error => console.error('Erreur Service Worker :',
error));
    }
</script>
```

Étape 5 : Servir le fichier en Debug (Fix specifique à .NET 8)

En développement (`dotnet watch`), le fichier `service-worker-assets.js` est généré dans le dossier `obj/` mais le projet d'hôte Serveur ne l'expose pas forcément sur son port par défaut, car il s'agit d'un système hybride. Pour que l'installation du cache ne tombe pas sur une erreur 404 pendant la programmation, ajoutez ce Middleware dans le `Program.cs` de votre **projet Serveur**, juste avant `app.Run()`.

```
// Serve the service worker manifest manually in development
app.MapGet("/service-worker-assets.js", async context =>
{
    var env =
context.RequestServices.GetRequiredService<IWebHostEnvironment>();
    // Ajustez finement le chemin selon votre structure (nommez bien
"MonApp.Client")
    var filePath = Path.Combine(env.ContentRootPath, "MonApp.Client",
"obj", "Debug", "net8.0", "service-worker-assets.js");

    if (File.Exists(filePath))
    {
        context.Response.ContentType = "application/javascript";
        await context.Response.SendFileAsync(filePath);
        return;
    }

    // Fallback silencieux en cas d'erreur
    context.Response.ContentType = "application/javascript";
    await context.Response.WriteAsync("self.assetsManifest = { assets: [],
version: '1' };");
});
```

(En production après un `dotnet publish`, les Web Static Assets gèrent ça automatiquement).

Étape 6 : Création des Pages et Fix de la Navigation Blazor

Blazor Web App tente d'optimiser les navigations entre les pages de menus (Enhanced Navigation). Mais **cette optimisation court-circuite le Service Worker**, et génère l'erreur **"Site Inaccessible"** lorsqu'on clique sur un lien en étant déconnecté du serveur SignalR.

1. **Créer une page Côté Client** (`MonApp.Client/Pages/Mapage.razor`).
2. **Rendu WebAssembly** : Exigez que cette page tourne en local, et non côté Serveur avec le tag `@rendermode`.

   ```
   @page "/mapage"
   @rendermode
   Microsoft.AspNetCore.Components.Web.RenderMode.InteractiveWebAssembly

   <h3>Je marche hors-ligne !</h3>
   ```

3. **Désactiver l'Enhanced Navigation pour ce lien**. Dans le fichier de votre menu de navigation (`NavMenu.razor`), il faut forcer le rechargement de la page (`data-enhance-nav="false"`) pour ce lien :

   ```
   <NavLink class="nav-link" href="mapage" data-enhance-nav="false">
       Ma Page WebAssembly
   </NavLink>
   ```

Félicitations ! Vous avez désormais les fondamentaux d'une base Blazor .NET 8 capable de servir des micro-applications intelligentes (Calculette, Chronos, etc.) de force et en parfaite autonomie réseau, grâce à ses Binaires `.dll` injectés via WebAssembly et gérés par le Service Worker.